

Mikroelektronische Schaltungen und Systeme

Vorlesung 12 Kombinatorische Logikschaltungen

Prof. Jürgen Becker
Institut für Technik der Informationsverarbeitung (ITIV)



Mikroelektronische Schaltungen und Systeme

Lect.1

Intro/CMOS Basics

Lect.2

Single Stage
Amplifiers

Lect.3

Noise

Lect.4

Differential Amplifiers

Lect.5

Current Mirrors

Lect.6

Frequency
Response

Lect.7

Feedback

Lect.8

Operational Amplifiers

Lect.9

Stability and
Frequency
Compensation

Lect.10

Switched-Capacitor
Circuits

Lect.11

Introduction to Data
Converters

Lect.12

Combinational Logic
Circuits

Lect.13

Sequential Circuits

Lect.14

Timing of Sequential
Circuits

Lect.15

Digital Building
Blocks

Lect.16

Memory

Lect.17

Technology
Mapping

Lect.18

FPGA

Lect.19

Physical Design I

Lect.20

Physical Design II

Lect.21

Q&A

1. Wdh: Boolesche Algebra, KV-Diagramme
2. Illegale und floating Zustände
3. Mehrstufige kombinatorische Logik
4. Kombinatorische Blöcke
5. Timing



Wdh: Boolesche Algebra, KV-Diagramme

1

Boolesche Algebra

Huntington-Algebra-Axiome

- Eine **algebraische Struktur** $(K, \vdash, \dashv)$ mit der **Menge** K und **zwei Operatoren** $\vdash$ und $\dashv$ ist eine **Huntington-Algebra**, wenn $(K, \vdash, \dashv)$ die folgenden **Axiome** erfüllt:

	Satz	Dual	Name
H1	$(a \vdash b) \in K$	$(a \dashv b) \in K$	Abgeschlossenheit
H2	$a \vdash b = a \vdash b$	$a \dashv b = a \dashv b$	Kommutativität
H3	$(a \vdash c) \dashv (b \vdash c) = c \vdash (a \dashv b)$	$(a \dashv c) \vdash (b \dashv c) = c \dashv (a \vdash b)$	Distributivität
H4	$I \vdash a = a$	$0 \dashv a = a$	Nullelement (NE)
H5	$a \vdash k = 0$	$a \dashv k = I$	Komplement

[2]

Mit: $a, b, c, k \in K$; $NeutralElement(\vdash) = I$; $NeutralElement(\dashv) = 0$; $k = \bar{a}$

Boolesche Algebra

Operatoren

- Sei: $\vdash = \wedge, \dashv = \vee, K = \{0,1\}, 0 = 0, 1 = 1$
- Definition: **Boolesche Algebra**: $BA = [\{0,1\}, \wedge, \vee, \bar{}, 0, 1]$

➤ **Boolesche Algebra**: Verknüpfungstabellen

$\wedge$	0	1
0	0	0
1	0	1

$\vee$	0	1
0	0	1
1	1	1

$\bar{a}$	
0	1
1	0

Boolesche Algebra

Dualität

- Das **Prinzip** der **Dualität** gilt für die **Boolesche Algebra**
 - Durch den **Austausch von $\wedge$ mit $\vee$ und *vice versa*** für einen Satz in der Booleschen Algebra entsteht der sogenannte **duale Satz**, der ebenfalls gilt.
 - Enthält der Satz die **Nullelemente 0 oder 1**, müssen diese **ebenfalls ausgetauscht** werden.
- Das Prinzip der **Dualität** ist wichtig, um Beweise zu vereinfachen und für die Verwendung der Booleschen Algebra („**Hälfte der Arbeit**“).

S. 97 [3]

Boolesche Algebra

Axiome der Booleschen Algebra

	Axiom		Dual	Name
A1	$B = 0 \text{ if } B \neq 1$	A1'	$B = 1 \text{ if } B \neq 0$	Binärfeld
A2	$\bar{1} = 0$	A2'	$\bar{0} = 1$	NICHT
A3	$0 \wedge 0 = 0$	A3'	$1 \vee 1 = 1$	UND/ODER
A4	$1 \wedge 1 = 1$	A4'	$0 \vee 0 = 0$	UND/ODER
A5	$0 \wedge 1 = 1 \wedge 0 = 0$	A5'	$1 \vee 0 = 0 \vee 1 = 1$	UND/ODER

S. 61 [1]

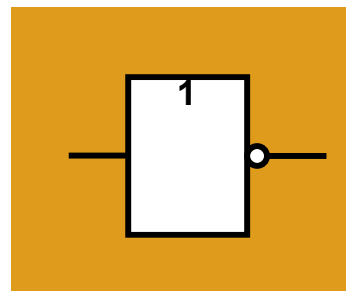
Boolesche Algebra

Axiome der Booleschen Algebra

	Axiom		Dual	Name
A1	$B = 0 \text{ if } B \neq 1$	A1'	$B = 1 \text{ if } B \neq 0$	Binärfeld
A2	$\bar{1} = 0$	A2'	$\bar{0} = 1$	NICHT
A3	$0 \wedge 0 = 0$	A3'	$1 \vee 1 = 1$	UND/ODER
A4	$1 \wedge 1 = 1$	A4'	$0 \vee 0 = 0$	UND/ODER
A5	$0 \wedge 1 = 1 \wedge 0 = 0$	A5'	$1 \vee 0 = 0 \vee 1 = 1$	UND/ODER

S. 61 [1]

NICHT



IEC 60617-12

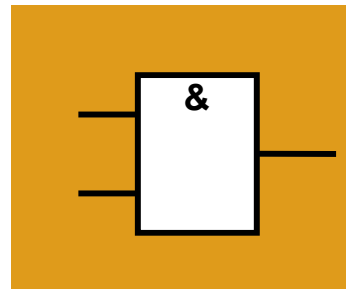
Boolesche Algebra

Axiome der Booleschen Algebra

	Axiom		Dual	Name
A1	$B = 0 \text{ if } B \neq 1$	A1'	$B = 1 \text{ if } B \neq 0$	Binärfeld
A2	$\bar{1} = 0$	A2'	$\bar{0} = 1$	NICHT
A3	$0 \wedge 0 = 0$	A3'	$1 \vee 1 = 1$	UND/ODER
A4	$1 \wedge 1 = 1$	A4'	$0 \vee 0 = 0$	UND/ODER
A5	$0 \wedge 1 = 1 \wedge 0 = 0$	A5'	$1 \vee 0 = 0 \vee 1 = 1$	UND/ODER

S. 61 [1]

UND



IEC 60617-12

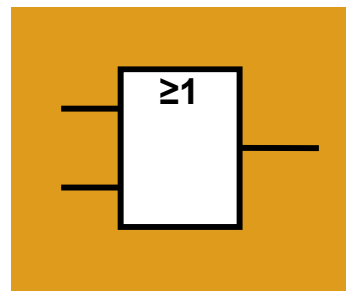
Boolesche Algebra

Axiome der Booleschen Algebra

	Axiom		Dual	Name
A1	$B = 0 \text{ if } B \neq 1$	A1'	$B = 1 \text{ if } B \neq 0$	Binärfeld
A2	$\bar{1} = 0$	A2'	$\bar{0} = 1$	NICHT
A3	$0 \wedge 0 = 0$	A3'	$1 \vee 1 = 1$	UND/ODER
A4	$1 \wedge 1 = 1$	A4'	$0 \vee 0 = 0$	UND/ODER
A5	$0 \wedge 1 = 1 \wedge 0 = 0$	A5'	$1 \vee 0 = 0 \vee 1 = 1$	UND/ODER

S. 61 [1]

ODER



IEC 60617-12

Boolesche Algebra

Boolesche Theoreme einer Variablen

Theorem		Dual		Name
T1	$B \wedge 1 = B$	T1'	$B \vee 0 = B$	Identität
T2	$B \wedge 0 = 0$	T2'	$B \vee 1 = 1$	Null-Element
T3	$B \wedge B = B$	T3'	$B \vee B = B$	Idempotenz
T4		$\bar{\bar{B}} = B$		<i>Involution</i>
T5	$B \wedge \bar{B} = 0$	T5'	$B \vee \bar{B} = 1$	Komplement

S. 62 [1]

Boolesche Algebra

Boolesche Theoreme mehrerer Variablen

Theorem		Dual		Name
T6	$B \wedge C = C \wedge B$	T6'	$B \vee C = C \vee B$	Kommutativität
T7	$(B \wedge C) \wedge D = B \wedge (C \wedge D)$	T7'	$(B \vee C) \vee D = B \vee (C \vee D)$	Assoziativität
T8	$(B \wedge C) \vee (B \wedge D) = B \wedge (C \vee D)$	T8'	$(B \vee C) \wedge (B \vee D) = B \vee (C \wedge D)$	Distributivität
T9	$B \wedge (B \vee C) = B$	T9'	$B \vee (B \wedge C) = B$	Abdeckung
T10	$(B \wedge C) \vee (B \wedge \bar{C}) = B$	T10'	$(B \vee C) \wedge (B \vee \bar{C}) = B$	Kombination
T11	$(B \wedge C) \vee (\bar{B} \wedge D) \vee (C \wedge D)$ $= B \wedge C \vee \bar{B} \wedge D$	T11'	$(B \vee C) \wedge (\bar{B} \vee D) \wedge (C \vee D)$ $= (B \vee C) \wedge (\bar{B} \vee D)$	Konsens
T12	$\overline{B_0 \wedge B_1 \wedge B_2 \dots}$ $= (\bar{B}_0 \vee \bar{B}_1 \vee \bar{B}_2 \dots)$	T12'	$\overline{B_0 \vee B_1 \vee B_2 \dots}$ $= (\bar{B}_0 \wedge \bar{B}_1 \wedge \bar{B}_2 \dots)$	De Morgan's Theorem

S. 62 [1]

Boolesche Algebra

Beispiel: Vereinfachung (Transformation)

$$(a \wedge b \wedge c) \vee (\bar{a} \wedge b) \vee (\overline{\bar{b} \vee (\bar{a} \wedge a)})$$

$$= (a \wedge b \wedge c) \vee (\bar{a} \wedge b) \vee (\overline{\bar{b} \vee 0})$$

$$= (a \wedge b \wedge c) \vee (\bar{a} \wedge b) \vee \bar{\bar{b}}$$

$$= (a \wedge b \wedge c) \vee (\bar{a} \wedge b) \vee b$$

$$= (a \wedge b \wedge c) \vee b$$

$$= b$$

$$(T5) \quad B \wedge \bar{B} = 0$$

$$(T1') \quad B \vee 0 = B$$

$$(T4) \quad \bar{\bar{B}} = B$$

$$(T9') \quad B \vee (C \wedge B) = B$$

$$(T9') \quad B \vee (C \wedge B) = B$$

**Komplementäre
Elemente**

Identität

Involution

Abdeckung

Abdeckung

Boolesche Algebra

Beispiel: Umwandlung in die disjunktive Form

$$((a \vee b) \wedge c) \vee d \vee (b \wedge c)$$

$$= (a \wedge c) \vee (b \wedge c) \vee d \vee (b \wedge c)$$

$$= (a \wedge c) \vee (b \wedge c) \vee d$$

(T8)

$$(B \wedge C) \vee (B \wedge D) \\ = B \wedge (C \vee D)$$

Distributivität

(T3')

$$B \vee B = B$$

Idempotenz

Boolesche Algebra

Beispiel: Transformation in eine Form ohne V-Operator

$$(a \wedge b) \vee (c \wedge d) \vee e$$

$$= \overline{\overline{(a \wedge b) \vee (c \wedge d) \vee e}}$$

$$= \overline{((\overline{a \wedge b}) \wedge (\overline{c \wedge d})) \wedge \bar{e}}$$

$$= \overline{(\overline{a \wedge b}) \wedge (\overline{c \wedge d})} \wedge \bar{e}$$

$$(T4) \quad \bar{\bar{B}} = B$$

Involution

$$(T12) \quad \overline{B_0 \wedge B_1 \wedge B_2 \dots} \\ = (\bar{B}_0 \vee \bar{B}_1 \vee \bar{B}_2 \dots)$$

**De Morgan
Theorem**

$$(T12) \quad \overline{B_0 \wedge B_1 \wedge B_2 \dots} \\ = (\bar{B}_0 \vee \bar{B}_1 \vee \bar{B}_2 \dots)$$

**De Morgans
Theorem**

Boolesche Algebra

Beweise für Theoreme

- In der **Booleschen Algebra** kann der Beweis eines Satzes mit einer **endlichen Anzahl** von **Variablen** mit der Methode der **vollständigen Induktion** durchgeführt werden
- Zeigt, dass der Satz für **alle möglichen Kombinationen** von Eingabewerten gilt
- Für diesen Anwendungsfall reicht meist eine **Wahrheitstabelle** aus

$$\overline{(A \wedge B)} \Leftrightarrow \bar{A} \vee \bar{B}$$

A	B	$\overline{(A \wedge B)}$	$\bar{A} \vee \bar{B}$
0	0	1	1
0	1	1	1
1	0	1	1
1	1	0	0

q.e.d.

Boolesche Algebra

Boolesche Funktion (auch logische Funktion, Schaltfunktion)

Definition:

Eine **Boolesche Funktion** definiert Tupel (x, y) für alle $x \in \{0,1\}^n$ mit $y \in \{0,1\}$



Boolesche Algebra

Boolesche Funktion (Schaltfunktion)

- Eine **Boolesche Funktion** kann als $y = f(X) = f(x_n, \dots, x_1, x_0)$ mit n **unabhängigen** Variablen $x_n, \dots, x_1, x_0$ und einer **abhängigen** Variablen y geschrieben werden

Beispiel: Sei $n = 2$ und f die Funktion für $\rightarrow$ (Implikation).

Zuordnungsregel

(x_1, x_0)	f	y
(0,0)	$\rightarrow$	1
(0,1)	$\rightarrow$	1
(1,0)	$\rightarrow$	0
(1,1)	$\rightarrow$	1

Wahrheitstabelle

x_1	x_0	y
0	0	1
0	1	1
1	0	0
1	1	1

Boolesche Algebra

Übersicht *Boolesche Funktionen* für 2-Input-Variablen

x_1	x_0	y
0	0	0
0	1	1
1	0	0
1	1	1

Übertragung

Leite Wert von x_0 .
 $y = x_0$

x_1	x_0	y
0	0	0
0	1	0
1	0	0
1	1	0

Null

Immer 0.
 $y = 0$

x_1	x_0	y
0	0	0
0	1	1
1	0	1
1	1	1

ODER

1, wenn x_0 oder x_1 gleich 1 sind.
 $y = x_1 \vee x_0$

x_1	x_0	y
0	0	1
0	1	0
1	0	1
1	1	1

Implikation

Wenn x_0 gleich 1 ist, dann leite x_1 , sonst 1.
 $y = x_1 \vee \neg x_0 = x_0 \rightarrow x_1$

x_1	x_0	y
0	0	0
0	1	0
1	0	1
1	1	1

Übertragung

Leite Wert von x_1 .
 $y = x_1$

x_1	x_0	y
0	0	1
0	1	1
1	0	1
1	1	1

Identität

Immer 1.
 $y = 1$

x_1	x_0	y
0	0	1
0	1	0
1	0	0
1	1	0

NOR

0, wenn x_0 oder x_1 gleich 1 sind.
 $y = \neg(x_1 \vee x_0)$

x_1	x_0	y
0	0	1
0	1	1
1	0	0
1	1	1

Implikation

Wenn x_1 gleich 1 ist, dann leite x_0 , sonst 1.
 $y = \neg x_1 \vee x_0 = x_1 \rightarrow x_0$

x_1	x_0	y
0	0	1
0	1	0
1	0	1
1	1	0

NICHT

Leite Wert von negiert x_0 .
 $y = \neg x_0$

x_1	x_0	y
0	0	0
0	1	0
1	0	0
1	1	1

UND

Nur 1, wenn x_0 und x_1 beide 1 sind.
 $y = x_1 \wedge x_0$

x_1	x_0	y
0	0	0
0	1	1
1	0	1
1	1	0

EX-OR

1, wenn x_0 oder x_1 gleich 1 sind, aber nicht beide.
 $y = x_1 \oplus x_0$

x_1	x_0	y
0	0	0
0	1	0
1	0	1
1	1	0

Inhibition

Wenn x_1 1 ist, dann leite negiert x_0 , sonst 0.
 $y = x_1 \wedge \neg x_0$

x_1	x_0	y
0	0	1
0	1	1
1	0	0
1	1	0

NICHT

Leite Wert von negiert x_1 .
 $y = \neg x_1$

x_1	x_0	y
0	0	1
0	1	1
1	0	1
1	1	0

NAND

Nur 0 wenn x_0 und x_1 beide 1 sind.
 $y = \neg(x_1 \wedge x_0)$

x_1	x_0	y
0	0	1
0	1	0
1	0	0
1	1	1

EX-NOR

0, wenn x_0 oder x_1 1 sind, aber nicht beide.
 $y = \neg(x_1 \oplus x_0)$

x_1	x_0	y
0	0	0
0	1	1
1	0	0
1	1	0

Inhibition

Wenn x_0 1 ist, dann leite negiert x_1 , sonst 0.
 $y = \neg x_1 \wedge x_0$

Boolesche Algebra

Übersicht *Boolesche Funktionen* für 2-Input-Variablen(Symbole)

x_1	x_0	y
0	0	0
0	1	1
1	0	0
1	1	1

Übertragung

Leite Wert von x_0 .

x_1	x_0	y
0	0	0
0	1	0
1	0	0
1	1	0

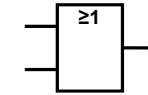
Null

Immer 0.

x_1	x_0	y
0	0	0
0	1	1
1	0	1
1	1	1

ODER

1, wenn x_0 oder x_1 gleich 1 sind.



x_1	x_0	y
0	0	1
0	1	0
1	0	1
1	1	1

Implikation

Wenn x_0 gleich 1 ist, dann leite x_1 , sonst 1.

x_1	x_0	y
0	0	0
0	1	0
1	0	1
1	1	1

Übertragung

Leite Wert von x_1 .

x_1	x_0	y
0	0	1
0	1	1
1	0	1
1	1	1

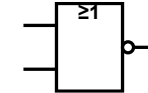
Identität

Immer 1.

x_1	x_0	y
0	0	1
0	1	0
1	0	0
1	1	0

NOR

0, wenn x_0 oder x_1 gleich 1 sind.



x_1	x_0	y
0	0	1
0	1	1
1	0	0
1	1	1

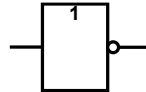
Implikation

Wenn x_1 gleich 1 ist, dann leite x_0 , sonst 1.

x_1	x_0	y
0	0	1
0	1	0
1	0	1
1	1	0

NICHT

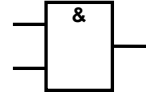
Leite Wert von negiert x_0 .



x_1	x_0	y
0	0	0
0	1	0
1	0	0
1	1	1

UND

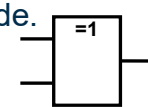
Nur 1, wenn x_0 und x_1 beide 1 sind.



x_1	x_0	y
0	0	0
0	1	1
1	0	1
1	1	0

EX-OR

1, wenn x_0 oder x_1 gleich 1 sind, aber nicht beide.



x_1	x_0	y
0	0	0
0	1	0
1	0	1
1	1	0

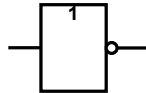
Inhibition

Wenn x_1 1 ist, dann leite negiert x_0 , sonst 0.

x_1	x_0	y
0	0	1
0	1	1
1	0	0
1	1	0

NOT

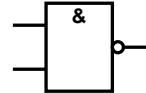
Leite Wert von negiert x_1 .



x_1	x_0	y
0	0	1
0	1	1
1	0	1
1	1	0

NAND

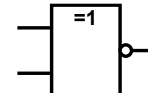
Nur 0 wenn x_0 und x_1 beide 1 sind.



x_1	x_0	y
0	0	1
0	1	0
1	0	0
1	1	1

EX-NOR

0, wenn x_0 oder x_1 1 sind, aber nicht beide.



x_1	x_0	y
0	0	0
0	1	1
1	0	0
1	1	0

Inhibition

Wenn x_0 1 ist, dann leite negiert x_1 , sonst 0.

Boolesche Algebra

Übersicht *Boolesche Funktionen* für 2-Input-Variablen

x_1	x_0	y
0	0	0
0	1	1
1	0	0
1	1	1

Übertragung

Leite Wert von x_0

x_1	x_0	y
0	0	0
0	1	0
1	0	1
1	1	1

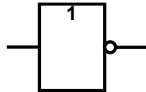
Übertragung

Leite Wert von x_1 .

x_1	x_0	y
0	0	1
0	1	0
1	0	1
1	1	0

NOT

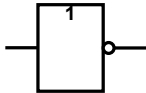
Pass negierter Wert von



x_1	x_0	y
0	0	1
0	1	1
1	0	0
1	1	0

NOT

Pass negierten Wert von



x_1	x_0	y
0	0	0
0	1	0
1	0	0
1	1	0

Null

Immer 0

x_1	x_0	y
0	0	1
0	1	1
1	0	1
1	1	1

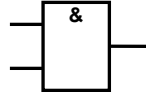
Identität

Immer 1

x_1	x_0	y
0	0	0
0	1	0
1	0	0
1	1	1

UND

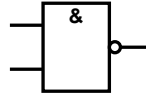
1 nur, wenn x_0 und x_1 beide 1 sind



x_1	x_0	y
0	0	1
0	1	1
1	0	1
1	1	0

NAND

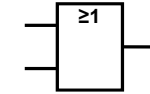
Nur 0 wenn x_0 und x_1 beide 1 sind.



x_1	x_0	y
0	0	0
0	1	1
1	0	1
1	1	1

ODER

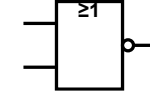
1, wenn x_0 oder x_1 gleich 1 sind.



x_1	x_0	y
0	0	1
0	1	0
1	0	0
1	1	0

NOR

0, wenn x_0 oder x_1 gleich 1 sind.



x_1	x_0	y
0	0	0
0	1	1
1	0	1
1	1	0

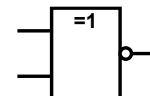
EX-OR

1 wenn x_0 oder x_1 1 sind, aber nicht beide.



x_1	x_0	y
0	0	0
0	1	0
1	0	0
1	1	1

1 sind, aber nicht beide.



x_1	x_0	y
0	0	1
0	1	0
1	0	1
1	1	1

Implikation

Wenn x_0 gleich 1 ist, dann leite x_1 , sonst 1.

x_1	x_0	y
0	0	1
0	1	1
1	0	0
1	1	1

Implikation

Wenn x_1 gleich 1 ist, dann leite x_0 , sonst 1.

x_1	x_0	y
0	0	0
0	1	1
1	0	1
1	1	0

Inhibition

Wenn x_1 1 ist, dann leite negiert x_0 , sonst 0.

x_1	x_0	y
0	1	1
1	0	0
1	1	0

Inhibition

Wenn x_0 1 ist, dann leite negiert x_1 , sonst 0.

Abhängig von keiner Variablen!

Boolesche Algebra

Übersicht *Boolesche Funktionen* für 2-Input-Variablen

x_1	x_0	y	Übertragung
0	0	0	Passwert von
0	1	1	
1	0	0	
1	1	1	

x_1	x_0	y	Übertragung
0	0	0	Wert übergeben von
0	1	0	
1	0	1	
1	1	1	

x_1	x_0	y	NOT
0	0	1	Pass negierter Wert von
0	1	0	
1	0	1	
1	1	0	

x_1	x_0	y	NOT
0	0	1	Pass negierten Wert von
0	1	1	
1	0	0	
1	1	0	

x_1	x_0	y
0	0	0
0	1	0
1	0	0
1	1	0

x_1	x_0	y
0	0	0
0	1	0
1	0	1
1	1	1

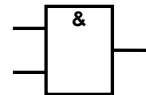
x_1	x_0	y
0	0	0
0	1	0
1	0	0
1	1	1

x_1	x_0	y
0	0	1
0	1	1
1	0	1
1	1	0

Nur von einer Variablen abhängig!

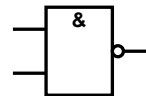
UND

1 nur, wenn x_0 und x_1 beide 1 sind



NAND

Nur 0 wenn x_0 und x_1 beide 1 sind.



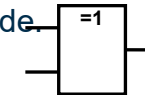
x_1	x_0	y
1	0	0
1	1	0

x_1	x_0	y
0	0	0
0	1	1
1	0	1
1	1	0

x_1	x_0	y
0	0	1
0	1	0
1	0	0
1	1	1

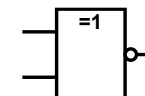
EX-OR

1, wenn x_0 oder x_1 gleich 1 sind, aber nicht beide.



EX-NOR

0, wenn x_0 oder x_1 1 sind, aber nicht beide.



x_1	x_0	y
0	0	1
0	1	0
1	0	1
1	1	1

Implikation

Wenn x_0 gleich 1 ist, dann leite x_1 , sonst 1.

x_1	x_0	y
0	0	1
0	1	1
1	0	0
1	1	1

Implikation

Wenn x_1 gleich 1 ist, dann leite x_0 , sonst 1.

x_1	x_0	y
0	0	0
0	1	0
1	0	1
1	1	0

Inhibition

Wenn x_1 1 ist, dann leite negiert x_0 , sonst 0.

x_1	x_0	y
0	0	0
0	1	1
1	0	0
1	1	0

Inhibition

Wenn x_0 1 ist, dann leite negiert x_1 , sonst 0.

Boolesche Algebra

Don't Care

- Es ist üblich, dass die *Boolesche Funktion* nicht jedem Input einen Wert zuweisen muss
- Bezeichnung: ` - ` für jeden **Wert**, der **0 oder 1** sein kann (manchmal auch **`X`**)
- Daher gibt es **drei Teilmengen**:

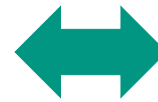
Menge von Nullen	$Z = \{Y y_i = 1\}$
Menge von Einsen	$E = \{Y y_i = 0\}$
Redundanzmenge	$R = \{Y y_i = -\}$

Boolesche Algebra

Don't Care

- Wenn eine **Ausgabe nicht** vom **Wert** einer **Eingabevariablen** abhängt, kann ein „**Don't Care**“ **-** geschrieben werden.

x_2	x_1	x_0	y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1



x_2	x_1	x_0	y
0	0	0	0
0	0	1	0
0	1	-	1
1	-	-	1

KV-Diagramme

Karnaugh-Diagramm

- Neben der tabellarischen Darstellung gibt es auch eine grafisch orientierte Darstellung: Das **KV-Diagramm**

Tabellarische Funktionsdarstellung

Index j im
8er-System:
-> Oktalzahlen

j	x_3	x_2	x_1	x_0	y
0	0	0	0	0	1
1	0	1	0	0	0
2	0	0	0	1	0
3	0	1	0	1	1
4	1	0	0	0	1
5	1	1	0	1	0
6	1	0	0	1	1
7	1	1	0	1	0
10	0	0	1	0	1
11	0	1	1	0	1
12	0	0	1	1	1
13	0	1	1	1	1
14	1	0	1	0	0
15	1	1	1	0	0
16	1	0	1	1	0
17	1	1	1	1	0



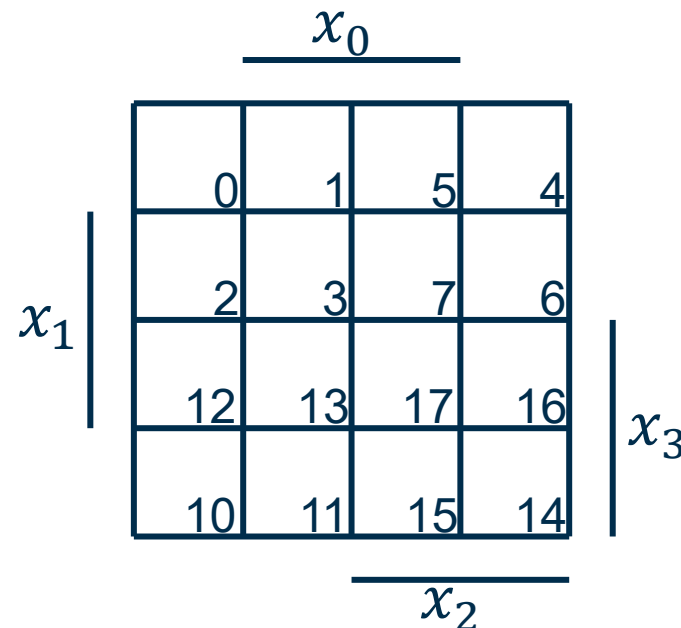
KV Diagramm

x_3, x_2 x_1, x_0	00	01	11	10
00	1 0	0 1	0 5	1 4
01	0 2	1 3	0 7	1 6
11	1 12	1 13	0 17	0 16
10	1 10	1 11	0 15	0 14

KV-Diagramme

KV-Diagramme Einschränkungen

- Es gibt einen Nachteil von *KV-Diagrammen*: Anzahl der Variablen von $n > 4$
- Das *Symmetriediagramm* ist für n Variablen erweiterbar [3]
 - Siehe Vorlesung Digitaltechnik

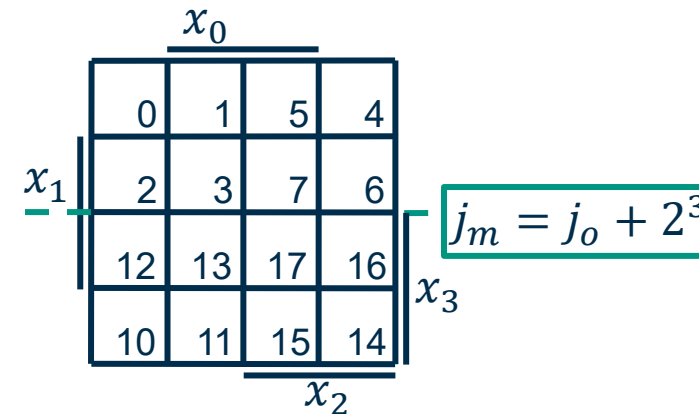
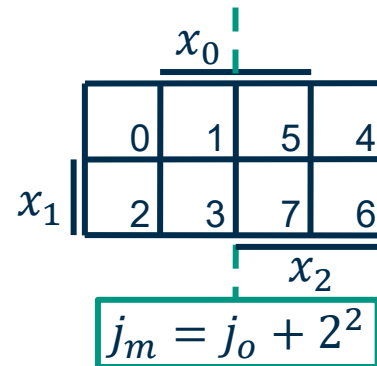
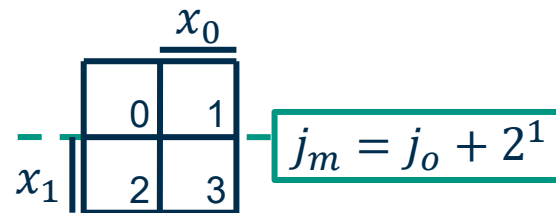
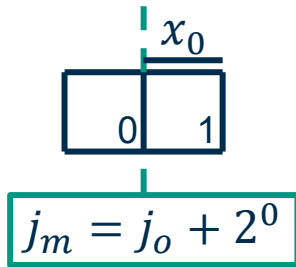


Index j im
8er-System:
-> Oktalzahlen

KV-Diagramme

KV-Diagramme Einschränkungen

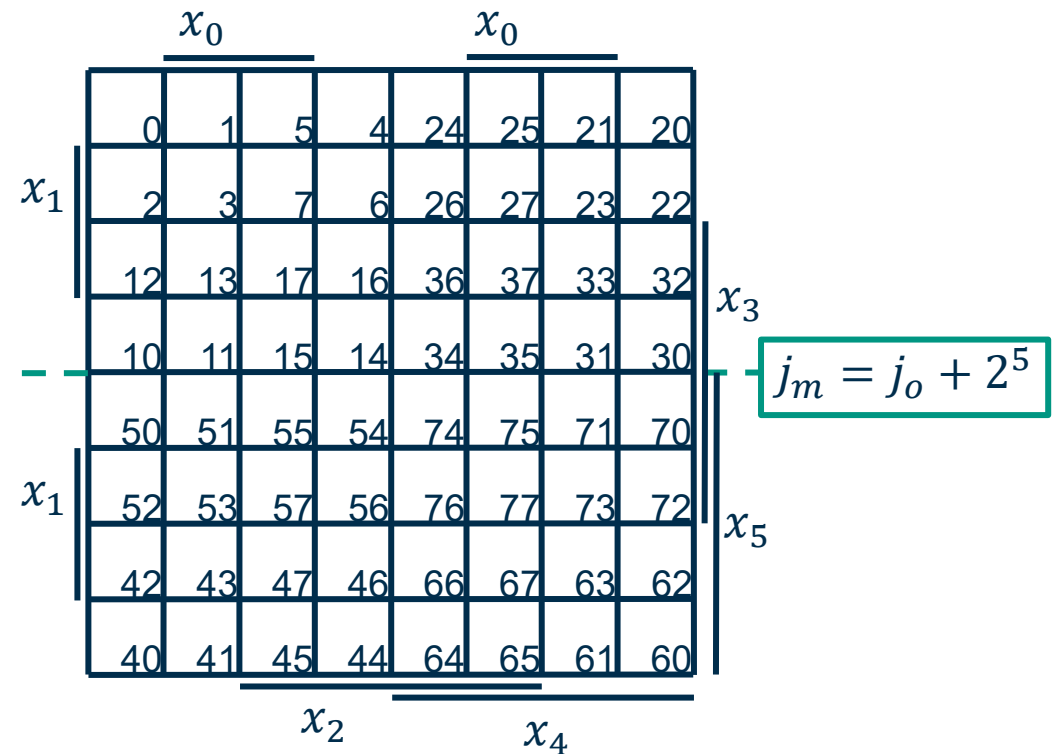
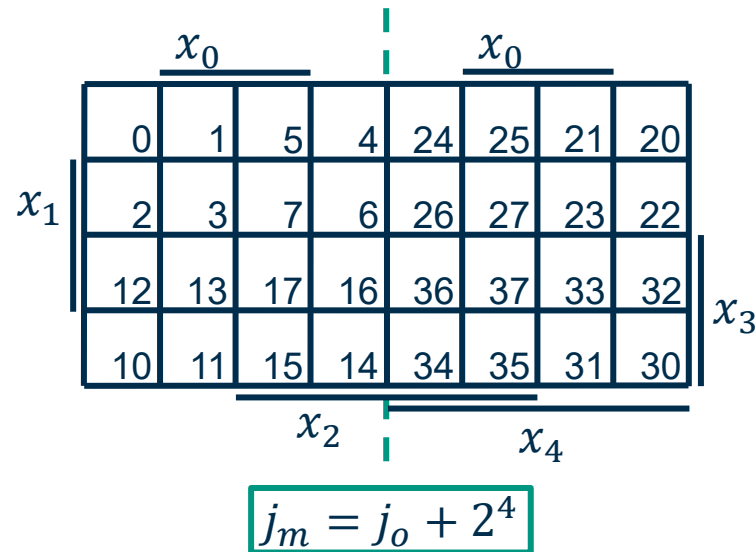
Erstellung von **Symmetriediagrammen**:



KV-Diagramme

KV-Diagramme Einschränkungen

Erstellung eines **Symmetriediagramms**:



KV-Diagramme

Logikminimierung mit *KV-Diagrammen*

- *KV-Diagramme* bieten eine **visuelle** Möglichkeit, einen **logischen Ausdruck** zu **vereinfachen**
- Dies geschieht durch die **Identifizierung** der **geringsten Anzahl** von **Primimplikanten**, indem sie mit „**Rechtecken**“ eingezeichnet werden

Regeln:

- **Alle 1en** einer Funktion müssen mit einem **Rechteck** überdeckt werden
- **Alle** mit einem **Rechteck** bedeckten **Quadrate** müssen **1en** enthalten
- Verwenden Sie eine **minimale Anzahl** von **Rechtecken**, um **alle 1en** abzudecken
- Jedes **Rechteck** muss **so groß** wie möglich sein
- Jedes **Rechteck** muss einen **rechteckigen Block** umfassen, der in jeder Richtung eine Potenz von 2 (d. h. 1, 2 oder 4 etc.) Quadrate umfasst

S. 62 [1]

KV-Diagramme

Logikminimierung mit *KV-Diagrammen*

Anmerkung:

- Ein *Rechteck* kann über die Ränder des *KV-Diagramms* *hinausreichen* und auf der gegenüberliegenden Seite wieder *hineinreichen*.
- Eine **1** in einem *KV-Diagramm* kann **mehrfach überdeckt** werden.

KV-Diagramme

Logikminimierung mit *KV-Diagrammen*

Beispiel:

x_3, x_2					
		00	01	11	10
x_1, x_0	00	1 ₀	0 ₁	0 ₅	1 ₄
	01	0 ₂	1 ₃	0 ₇	1 ₆
	11	1 ₁₂	1 ₁₃	0 ₁₇	0 ₁₆
	10	1 ₁₀	1 ₁₁	0 ₁₅	0 ₁₄

- Alle 1en müssen mit einem *Rechteck* überdeckt werden
- Alle mit einem *Rechteck* bedeckten *Quadrate* müssen 1en enthalten

KV-Diagramme

Logikminimierung mit *KV-Diagrammen*

Beispiel:

x_3, x_2					
		00	01	11	10
x_1, x_0	00	1 ₀	0 ₁	0 ₅	1 ₄
	01	0 ₂	1 ₃	0 ₇	1 ₆
	11	1 ₁₂	1 ₁₃	0 ₁₇	0 ₁₆
	10	1 ₁₀	1 ₁₁	0 ₁₅	0 ₁₄

- Verwenden Sie eine **minimale** Anzahl von **Rechtecken**, um **alle 1en** abzudecken
- Jedes **Rechteck** muss **so groß** wie möglich sein
- Jedes **Rechteck** muss einen **rechteckigen Block** umfassen, der in jeder Richtung eine **Potenz von 2** (d. h. 1, 2 oder 4 etc.) **Quadrate** umfasst.

KV-Diagramme

Logikminimierung mit *KV-Diagrammen*

Beispiel:

x_3, x_2					
		00	01	11	10
x_1, x_0	00	1 ₀	0 ₁	0 ₅	1 ₄
	01	0 ₂	1 ₃	0 ₇	1 ₆
	11	1 ₁₂	1 ₁₃	0 ₁₇	0 ₁₆
	10	1 ₁₀	1 ₁₁	0 ₁₅	0 ₁₄

- Jedes *Rechteck* muss so groß wie möglich sein
- Ein *Rechteck* kann über die Ränder des *KV-Diagramms* hinausreichen und auf der gegenüberliegenden Seite wieder hineinreichen

KV-Diagramme

Logikminimierung mit *KV-Diagrammen*

Beispiel:

x_3, x_2		x_1, x_0			
		00	01	11	10
00	00	1 ₀	0 ₁	0 ₅	1 ₄
	01	0 ₂	1 ₃	0 ₇	1 ₆
	11	1 ₁₂	1 ₁₃	0 ₁₇	0 ₁₆
	10	1 ₁₀	1 ₁₁	0 ₁₅	0 ₁₄

- Jedes *Rechteck* muss so groß wie möglich sein
- Eine 1 in einem *KV-Diagramm* kann mehrfach überdeckt werden

KV-Diagramme

Logikminimierung mit *KV-Diagrammen*

x_3, x_2 x_1, x_0		x_3, x_2			
		00	01	11	10
00	1 ₀	0 ₁	0 ₅	1 ₄	
01	0 ₂	1 ₃	0 ₇	1 ₆	
11	1 ₁₂	1 ₁₃	0 ₁₇	0 ₁₆	
10	1 ₁₀	1 ₁₁	0 ₁₅	0 ₁₄	



x_3, x_2 x_1, x_0		x_3, x_2			
		00	01	11	10
00	1 ₀	0 ₁	0 ₅	1 ₄	
01	0 ₂	1 ₃	0 ₇	1 ₆	
11	1 ₁₂	1 ₁₃	0 ₁₇	0 ₁₆	
10	1 ₁₀	1 ₁₁	0 ₁₅	0 ₁₄	

$$\begin{aligned}
 \text{DNF} = & (\overline{x_3} \overline{x_2} \overline{x_1} \overline{x_0}) \vee (x_3 \overline{x_2} \overline{x_1} \overline{x_0}) \vee \\
 & (\overline{x_3} x_2 \overline{x_1} x_0) \vee (x_3 \overline{x_2} \overline{x_1} x_0) \vee \\
 & (\overline{x_3} \overline{x_2} x_1 x_0) \vee (\overline{x_3} x_2 x_1 x_0) \vee \\
 & (\overline{x_3} \overline{x_2} x_1 \overline{x_0}) \vee (\overline{x_3} x_2 x_1 \overline{x_0})
 \end{aligned}$$

$$\begin{aligned}
 \text{DMF} = & (\overline{x_2} \overline{x_1} \overline{x_0}) \vee \\
 & (\overline{x_3} x_2 x_0) \vee (x_3 \overline{x_2} \overline{x_1}) \vee \\
 & (\overline{x_3} x_1)
 \end{aligned}$$

VON DER LOGIK ZU DEN „GATES“ (Gatter)

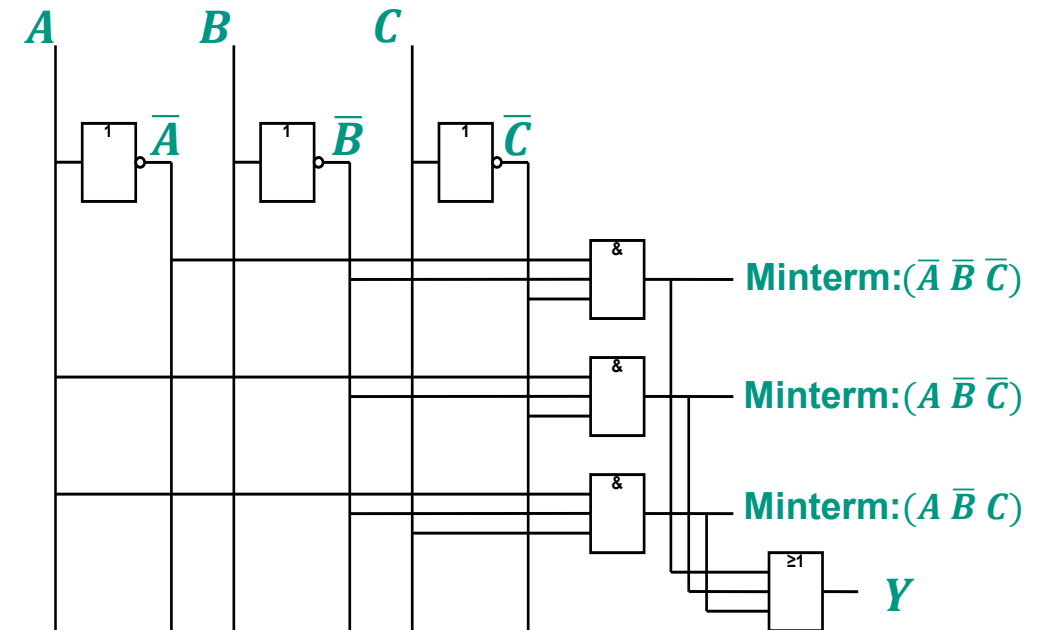
Schaltplan

- Ein **Schaltplan** ist eine **Darstellungsform** für eine **digitale Schaltung**, die **Elemente** und die **Verbindungen** zwischen ihnen zeigt

Beispiel: Logik in **disjunktiver Normalform**

$$Y = (\bar{A} \bar{B} \bar{C}) \vee (A \bar{B} \bar{C}) \vee (A \bar{B} C)$$

Referenz S. 67 [1]



VON DER LOGIK ZU DEN „GATES“ (Gatter)

Schematische Darstellung: *Leitungen*

- Im Allgemeinen gelten die folgenden Richtlinien:
 - *Leitungen* verbinden sich an einer T-Verbindung
 - *Leitungen* verbinden sich an einem Punkt
 - *Leitungen*, die sich ohne Punkt kreuzen, sind nicht verbunden





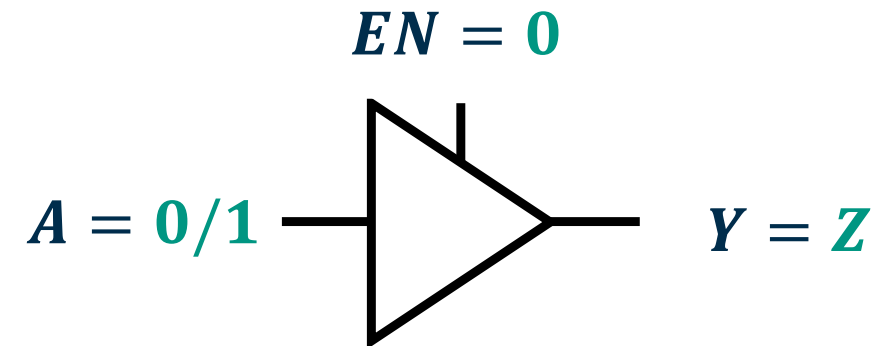
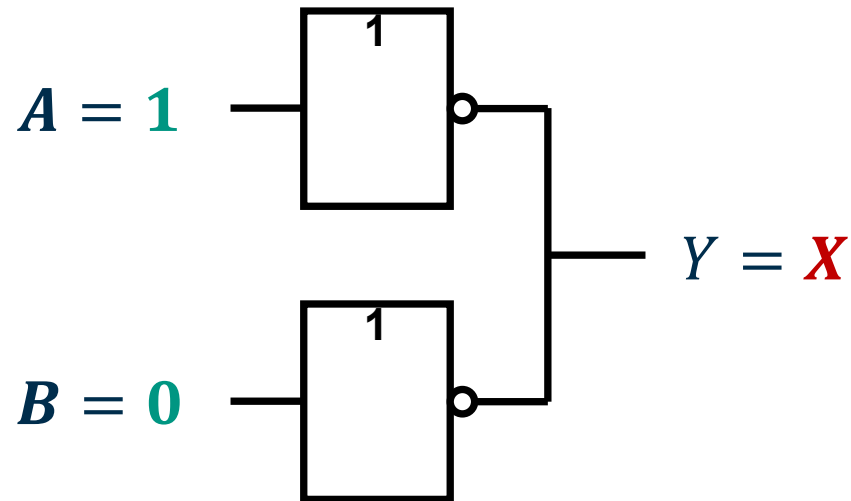
Illegale und floating Zustände

2

Illegale und floating Zustände

Einführung

- Die **Boolesche Algebra** mit ihrer **binären Größe {0,1}** reicht für reale Schaltungen nicht aus. **Reale Schaltungen** können auch **illegale** und **floating** Zustände übertragen, die durch **X** und **Z** dargestellt werden

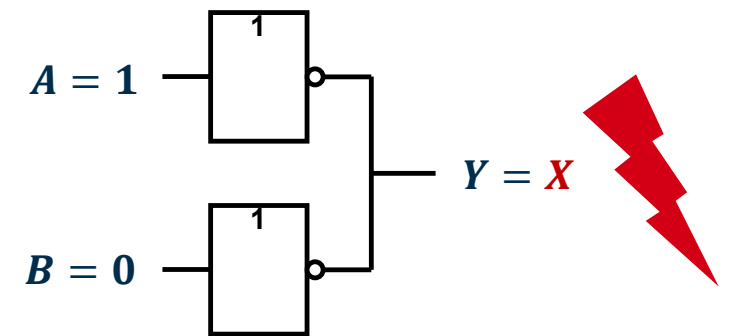


Referenz S. 73 [1]

Illegale und floating Zustände

Illegaler Wert: **X**

- Der Wert **X** weist auf einen **unzulässigen** oder **unbekannten** Wert hin
- Ein häufiger Fall, in dem dies auftritt, ist, wenn ein **Schaltungsknoten gleichzeitig** von **verschiedenen Treibern** auf **0** und **1** getrieben wird
- Dieser Fall wird als „**Contention**“ bezeichnet und gilt als **Fehler** und muss verhindert werden.
- In diesem Fall liegt der **Wert zwischen 0** und V_{DD}
- **Contention** kann zu hohen Strömen führen, die Schäden verursachen können (ähnlich wie bei einem **Kurzschlussfehler**)



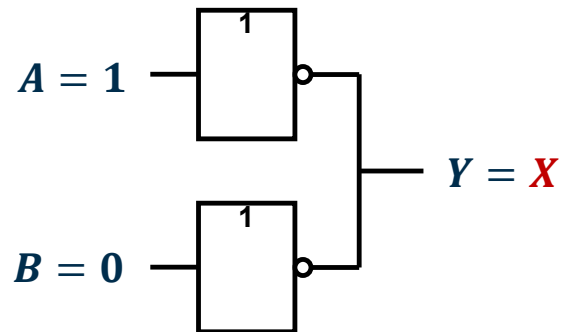
Referenz S. 73 [1]

Illegale und floating Zustände

Illegal Zustand: **X**

Anmerkung:

- **X** wird auch verwendet, um „*don't-care*“ Werte in **Wahrheitstabellen** anzuzeigen
- Wenn **X** in **Wahrheitstabellen** erscheint, steht es für einen *don't-care* Wert einer **Variablen**. Wenn **X** in einer **Schaltung** oder **Waveform** Darstellung verwendet wird, weist es auf einen *unbekannten* oder *illegalen* Wert hin



$\neq$

x_2	x_1	x_0	y
0	0	0	0
0	0	1	0
0	1	X	1
1	X	X	1

Referenz S. 72 [1]

Illegale und floating Zustände

Floating Zustand: **Z**

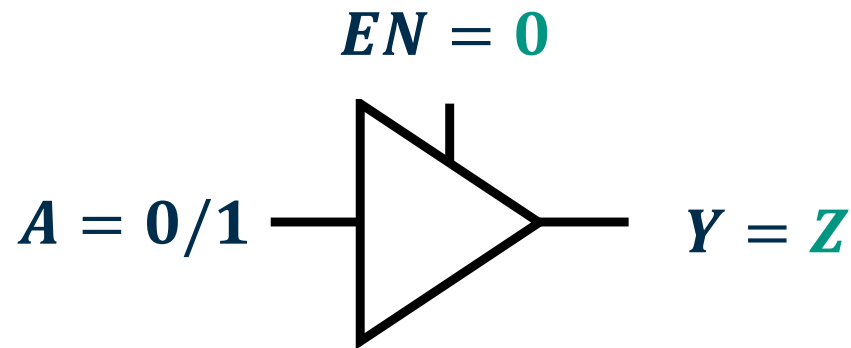
- Der Wert **Z** kennzeichnet einen **Schaltungsknoten**, der **weder** auf *HIGH* noch auf *LOW* gesetzt ist. Der Knoten wird als **schwebend** oder **hochohmig** bezeichnet
- Ein **schwebender Knoten** ist nicht immer ein **Fehler**, vorausgesetzt, dass eine andere Quelle den Knoten ansteuert, wenn der Wert des Knotens für einen Vorgang wichtig ist
 - Eine **gängige Methode** zur Erzeugung eines **schwebenden Knotens** besteht darin, die **Schaltung nicht an Power** anzuschließen
 - Auch ein **nicht angeschlossener Input** ist **floating**
- Eine abhängige Schaltung kann sich fehlerhaft verhalten, da der schwebende Knoten von der Schaltung zufällig als 0 oder 1 gelesen werden kann.

Referenz S. 73 [1]

Illegale und floating Zustände

Floating Zustand: Z

- Der **Tristate-Buffer** hat einen gültigen Wert **Z** als Ausgabe für den Fall, dass der Tristate-Buffer **nicht aktiviert** ist
- Wenn der **Enable-Input 0** ist, darf der **Ausgang Y floaten**



EN	A	Y
0	0	Z
0	1	Z
1	0	0
1	1	1

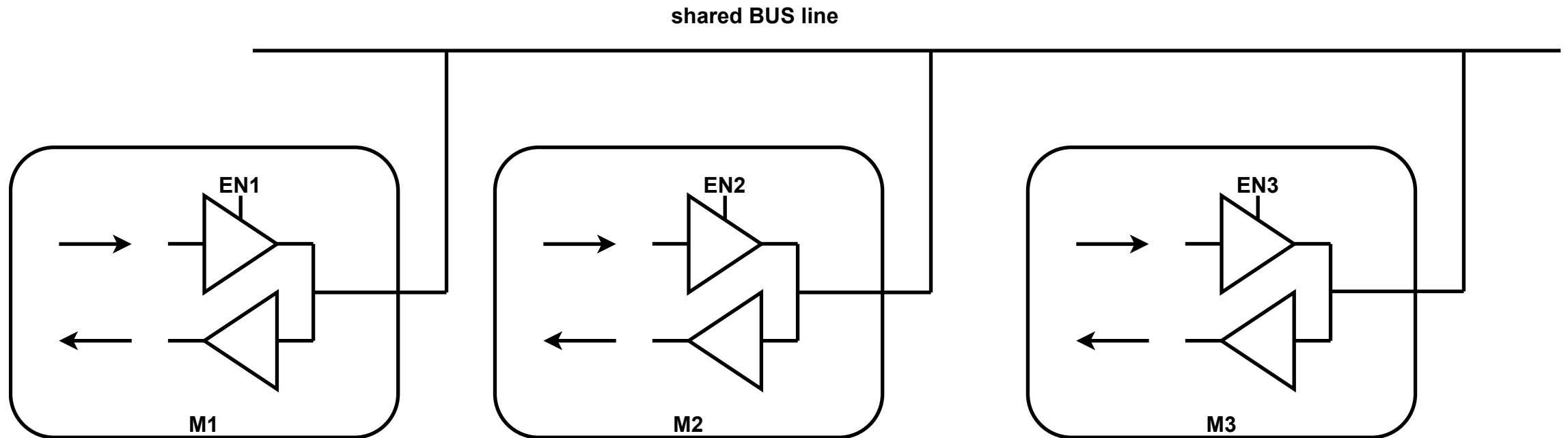
Referenz S. 73 [1]

Illegale und floating Zustände

Floating Zustand: Z

Beispiel:

- **Tristate-Puffer**, der **mehrere Teilnehmer** mit derselben **gemeinsamen Leitung** verbindet



Referenz S. 75 [1]

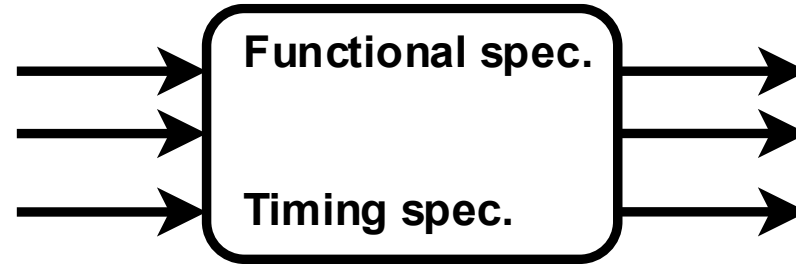


Mehrstufige kombinatorische Logik

2

Mehrstufige kombinatorische Logik

Einführung:

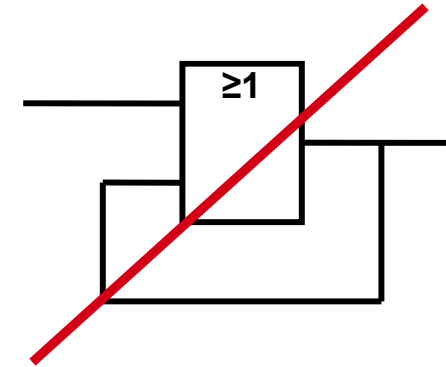


Referenz S. 55 [1]

- Eine **Schaltung** ist ein Netzwerk, das **diskrete Werte** verarbeitet
- Eine **Schaltung** besteht aus:
 - Einem oder mehreren **diskreten Eingängen**
 - Einem oder mehreren **diskreten Ausgängen**
 - einer **Funktion-Spezifikation**, die die **Beziehung** zwischen **Eingängen** und **Ausgängen** beschreibt
 - Eine **Timing-Spezifikation**, die die **Verzögerung** zwischen der **Änderung** der **Eingänge** und der **Reaktion** der **Ausgänge** beschreibt

Mehrstufige kombinatorische Logik

Definition:



Referenz S. 57 [1]

- Eine **Schaltung** ist *kombinatorisch*, wenn:
- Sie aus miteinander **verbundenen Schaltungselementen** besteht, die *selbst kombinatorisch* sind
- Jeder **Knoten** der **Schaltung** entweder ein **externer Input** der Schaltung ist oder mit genau einem **Ausgangsanschluss** eines anderen **Schaltungselements** verbunden ist (**keine Contention**)
- Die **Schaltung** enthält **keine zyklischen Pfade**: Jeder Pfad durch die Schaltung durchläuft **jeden Schaltungsknoten höchstens einmal**

Mehrstufige kombinatorische Logik

Einführung:

- Logik in der *Disjunktiven Normalform (DNF)* (auch *Summenform*) wird als **zweistufige Logik** bezeichnet, da sie aus **zwei Stufen** besteht:
 - *AND*-Gatter-Stufe
 - *ODER*-Gatter-Stufe
- Häufig werden **Schaltungen** mit **mehreren Stufen** entworfen, die als *mehrstufige kombinatorische* **Schaltungen** bezeichnet werden.

Mehrstufige kombinatorische Logik

Optimierung der Logik:

- Die **Optimierung** der **Logik** muss unter Berücksichtigung vieler **unterschiedlicher Parameter** erfolgen:
 - Anzahl der **Gatter**
 - Schnellste **Logik**
 - Kürzeste **Entwicklungszeit**
 - Geringste **Kosten**
 - Geringster **Stromverbrauch**
- **Beachten Sie**, dass eine für eine **Technologie optimierte Logik** nicht unbedingt auch für eine andere Technologie eine gute Lösung darstellt. Beispielsweise sind **NAND-** und **NOR-Gatter** für **CMOS** **effizienter** als **AND-** und **OR-Gatter**.

Mehrstufige kombinatorische Logik

Bubble Pushing

- „**Bubble Pushing**“ ist eine **Technik** zum **Neuzeichnen** von **Schaltungen**, die aus mehreren **Ebenen** von **NAND-** und **NOR-Gattern** bestehen. Dies soll die **Verständlichkeit** für den Leser verbessern. **Bubble** bezieht sich auf den **Kreis** auf einer **Gate-Zeichnung**, der angibt, ob ein **Input** oder **Output negiert** wird:
 1. **Beginnen** Sie am **Output** der Schaltung und arbeiten Sie sich **zu den Inputs** vor
 2. **Schieben** Sie alle **Blasen** in Richtung der **Eingänge**, sodass Sie den **Ausgang nicht-negiert anstelle** des **Komplements** lesen können (z. B. **Y** anstelle von $\bar{Y}$)
 3. **Zeichnen** Sie jedes **Gatter** so, dass sich die **Bubbles** aufheben:
 - Wenn das aktuelle **Gatter eine Input-Bubble** hat, zeichnen Sie das vorhergehende **Gatter** mit einer **Output-Bubble**
 - Wenn das aktuelle **Gatter keine Input-Bubble** hat, zeichnen Sie das vorhergehende **Gatter ohne Output-Bubble**

Mehrstufige kombinatorische Logik

Bubble Pushing

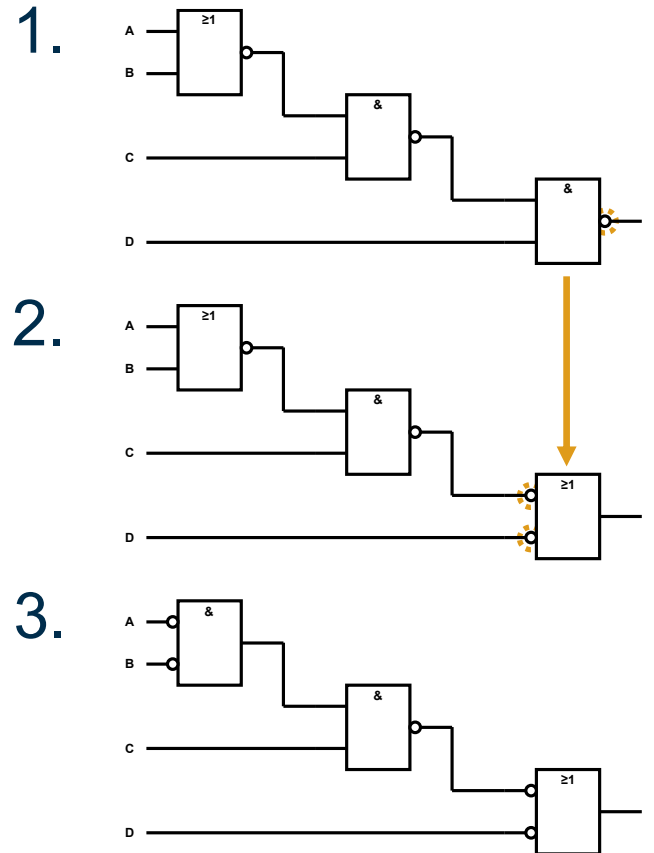
■ Regeln:

1. Beginnen Sie am **Ausgang** der **Schaltung** und arbeiten sich zum **Eingang** vor
2. Schieben Sie alle **Bubbles** in **Richtung** der **Inputs**, sodass ein **identitärer Output** anstelle des Invertierten gelesen werden kann (z. B. **Y anstelle** von $\bar{Y}$)
3. Zeichnen Sie jede **Verbindung** so, dass sich die **Bubbles aufheben**:
 - Wenn das aktuelle **Gatter** eine **Input-Bubble** hat,
zeichne das **vorhergehende** Gatter mit einer **Ausgangs-Bubble**
 - Wenn nicht, dann
zeichne das **vorhergehende** Gatter ohne **Ausgangs-Bubble**

Mehrstufige kombinatorische Logik

Bubble Pushing (Blasenverschiebung)

Beispiel:



➤ Schieben Sie die *Blase* vom *Output* zum *Input*.

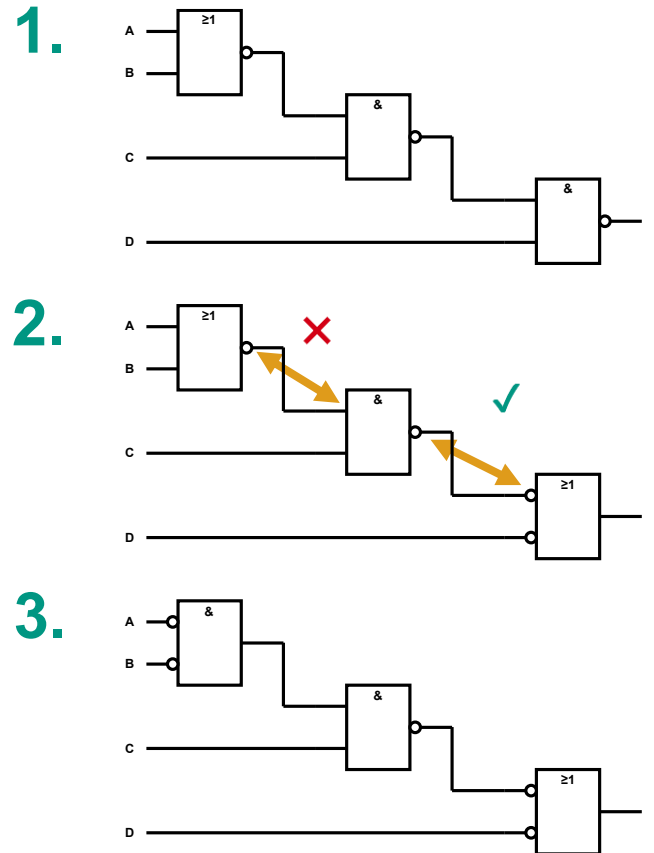
Das *Gate* ändert sich gemäß dem *De-Morgan-Theorem* (T12).

Referenz S. 72 [1]

Mehrstufige kombinatorische Logik

Bubble Pushing (Blasenverschiebung)

Beispiel:



➤ Schieben Sie die *Blase* vom *Output* zum *Input*.

Das *Gate* ändert sich gemäß dem *De-Morgan-Theorem* (T12)

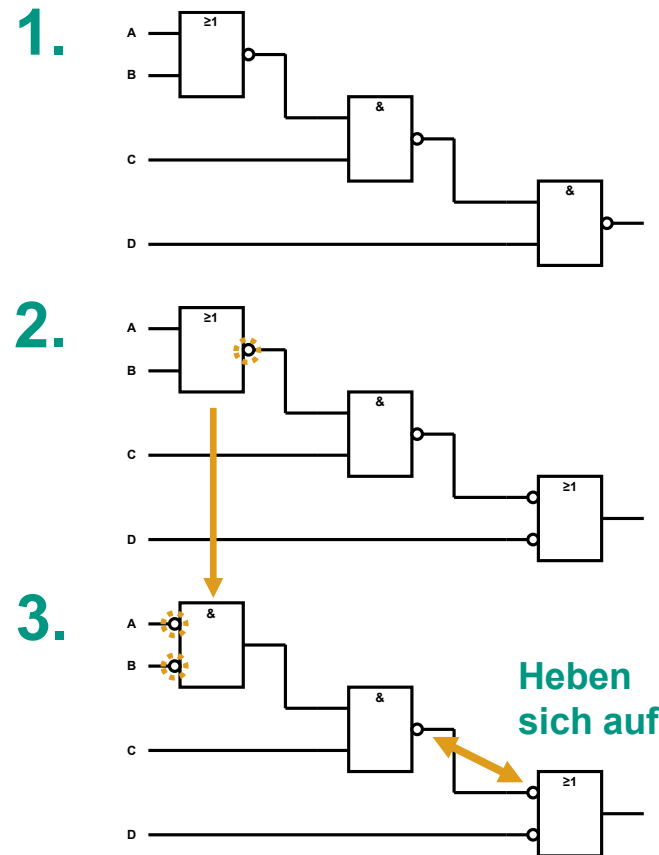
➤ Zeichnen Sie jedes *Gate* so, dass sich die *Blasen* aufheben

Referenz S. 72 [1]

Mehrstufige kombinatorische Logik

Bubble Pushing (Blasenverschiebung)

Beispiel:



➤ Schieben Sie die *Blase* vom *Output* zum *Input*.

Das *Gate* ändert sich gemäß dem *De-Morgan-Theorem* (T12)

➤ Zeichnen Sie jedes *Gate* so, dass sich die *Blasen* aufheben

$$Y = \overline{A}\overline{B}C \vee \overline{D}$$

S. 72 [1]



Kombinatorische Blöcke

3

Kombinatorische Blöcke

Einführung:

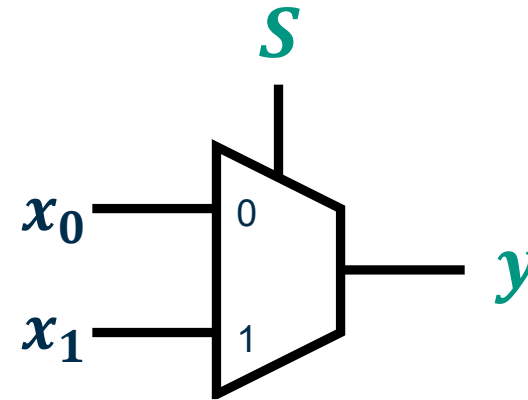
- Um **komplexere Systeme** zu bauen, wird die Logik in **höherwertigen Blöcken** gruppiert. In diesem Kapitel werden die **folgenden Blöcke** vorgestellt:
 - *Multiplexer*
 - *Decoder*

Kombinatorische Blöcke

Multiplexer

- **Multiplexer** sind gängige *kombinatorische Schaltungen*, die anhand eines **Auswahlsignals** (*select*) den **Ausgang** aus ihren **Eingängen** **auswählen**

S	x_1	x_0	y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

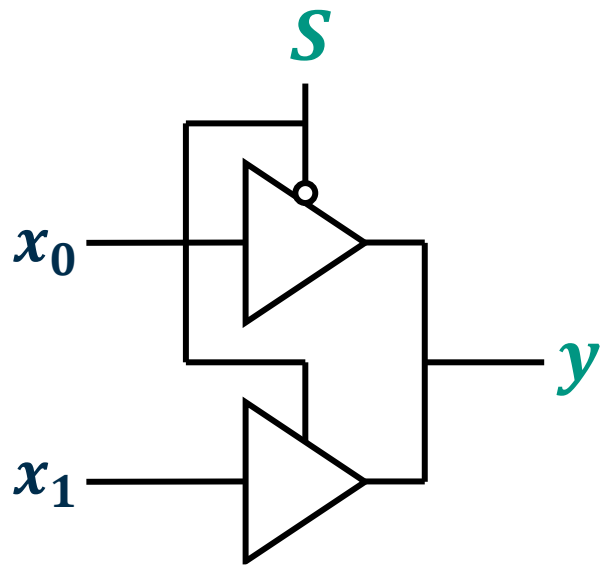


Referenz: 2:1-Multiplexer S. 83 [1]

Kombinatorische Blöcke

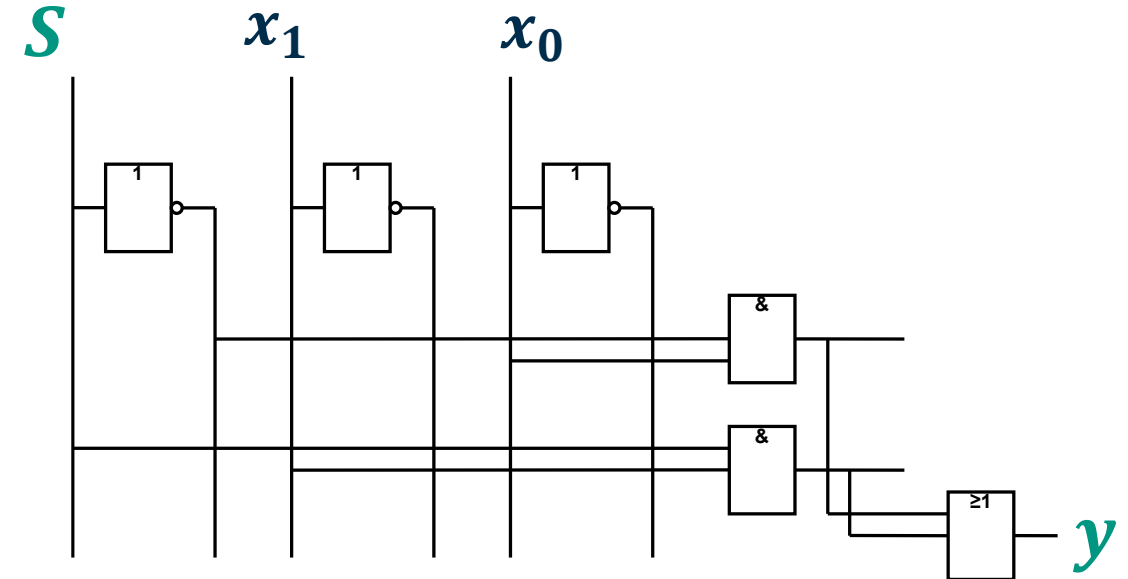
Multiplexer

- Es werden **zwei** mögliche *Implementierungen* für den **2:1-Multiplexer** vorgestellt. Diese Ansätze sind auch auf **x:1-Multiplexer** übertragbar



Implementierung mit *Tristate-Buffer*

Referenz S. 84 [1]



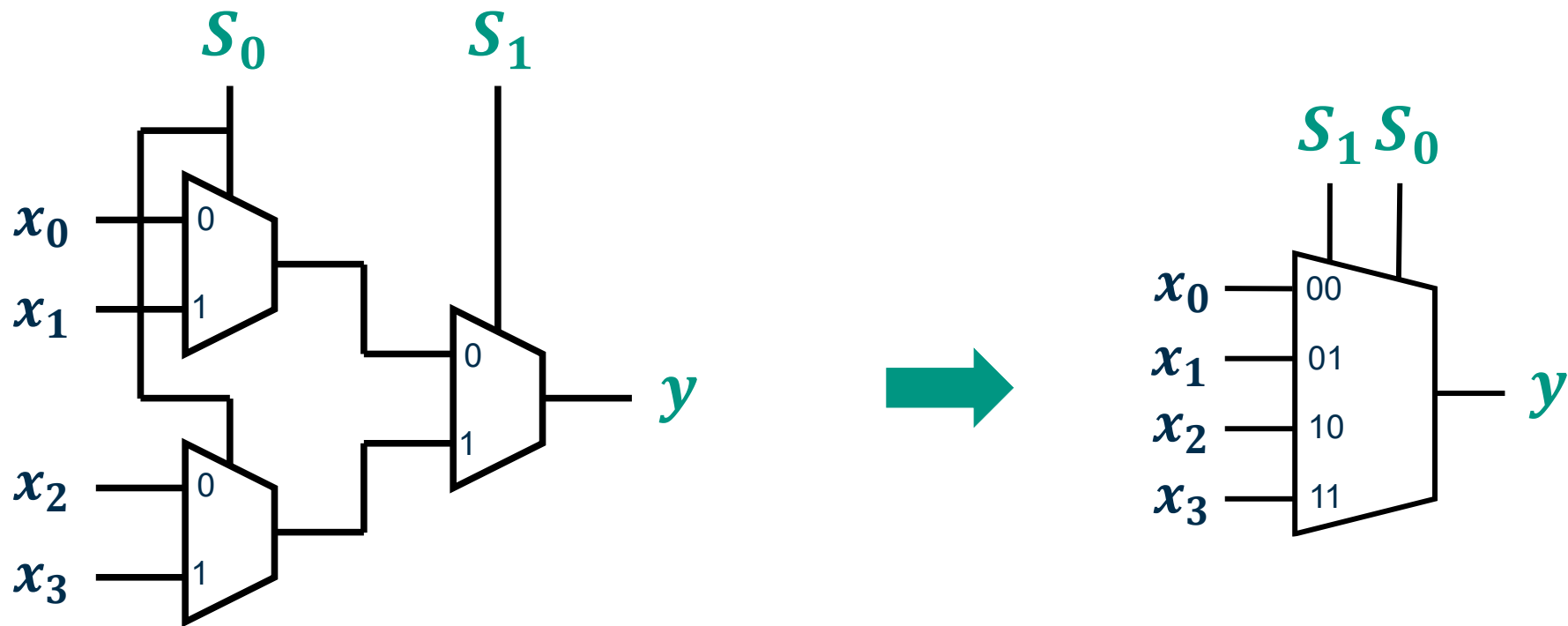
Implementierung mit *zweistufiger Logik*

Referenz S. 84 [1]

Kombinatorische Blöcke

Multiplexer x:1 (hierarchisch)

- Mit **2:1-Multiplexern** lassen sich durch hierarchische Anordnung breitere **x:1-Multiplexer** aufbauen



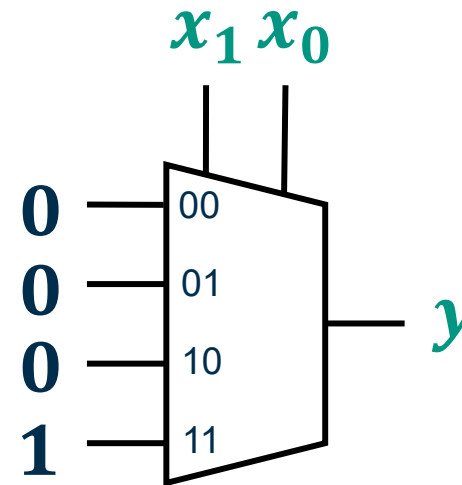
Kombinatorische Blöcke

Lookup Table (*Multiplexer*)

- *Multiplexer* können als *Lookup Table* verwendet werden, um *verschiedene Logikfunktionen* zu realisieren

Beispiel: *4:1-Multiplexer* als Lookup Table verwendet, um die AND-Funktion mit zwei Eingängen zu erzeugen.

x_1	x_0	y
0	0	0
0	1	0
1	0	0
1	1	1



Referenz S. 84 [1]

Kombinatorische Blöcke

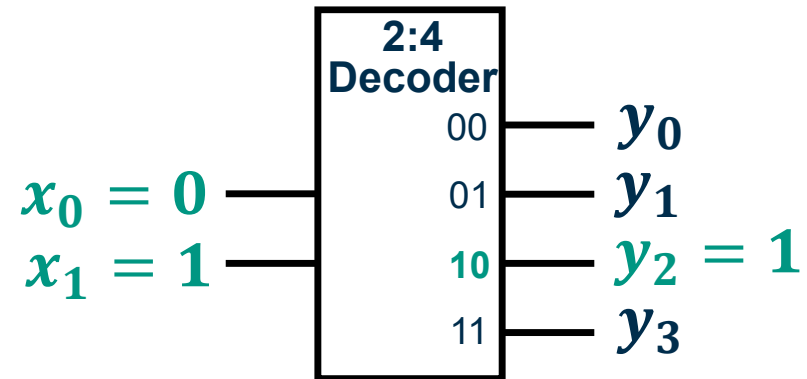
Decoder

- Ein **Decoder** hat **N Eingänge** und **2^N Ausgänge**
Er setzt genau **einen Ausgang** auf **1**, abhängig von der Inputkombination
Die **Ausgänge** werden als **one-hot** bezeichnet

Beispiel:

2:4-Decoder

x_1	x_0	y_3	y_2	y_1	y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	1



Referenz S. 87 [1]

Kombinatorische Blöcke

Decoder

- Ein *Decoder* in Kombination mit *ODER-Gattern* kann ebenfalls zur Realisierung *logischer Funktionen* verwendet werden

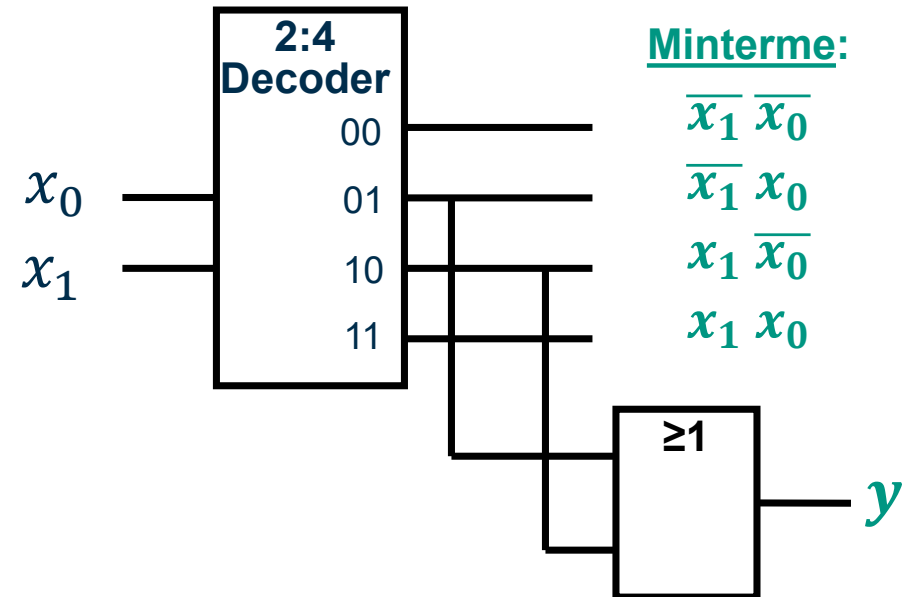
Beispiel:

x_1	x_0	y
0	0	0
0	1	1
1	0	1
1	1	0



$$y = \overline{x_1}x_0 \vee x_1 \overline{x_0}$$

$$y = x_1 \oplus x_0$$



Referenz S. 87 [1]



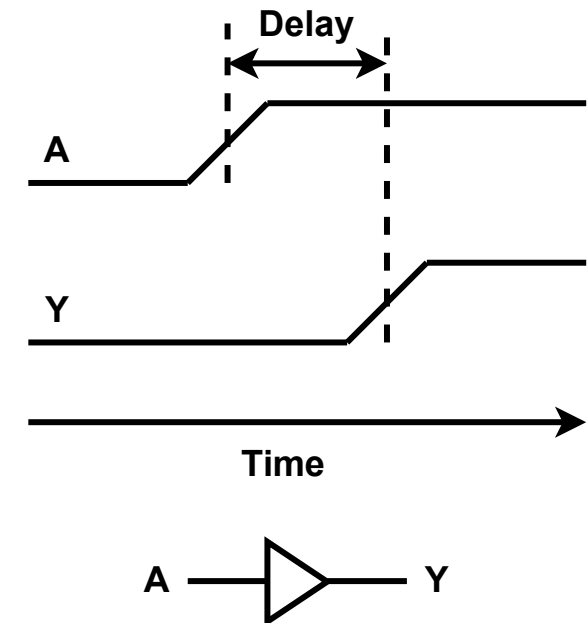
Timing

3

Timing

Einführung:

- Der **Ausgang** einer **Schaltung** benötigt **Zeit**, um auf eine Änderung der Eingabe zu reagieren. Dies wird als **Delay** bezeichnet. Die **Verzögerung** wird durch **Messen der Übergangszeiten** der **Eingabe** und der **Ausgabe** berechnet
 - Ein Übergang von **LOW zu HIGH** wird als **steigende Flanke** (*rising edge*) bezeichnet
 - Ein Übergang von **HIGH zu LOW** wird als **fallende Flanke** (*falling edge*) bezeichnet
- Gemessen** werden immer die **50 %-Schwellen** der **Übergänge**. Die **50 %-Schwelle** ist der **Punkt**, an dem sich das **Signal** während des Übergangs auf **halbem Weg** zwischen dem **LOW-** und dem **HIGH-Wert** befindet

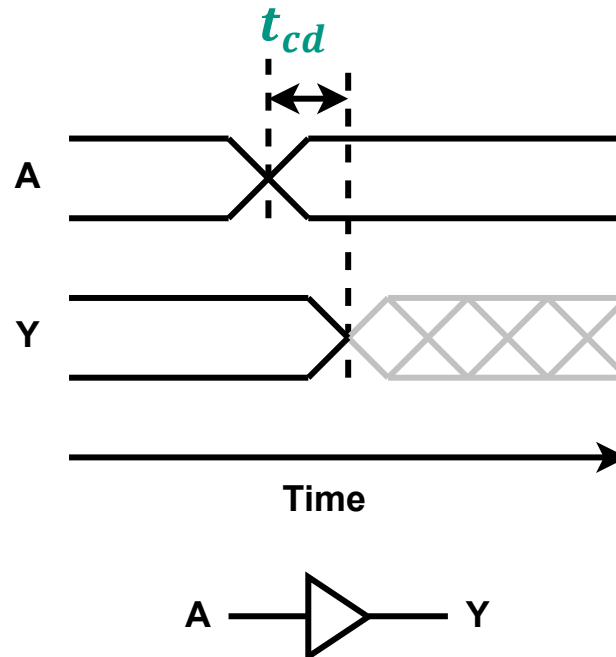


Referenz S. 88 [1]

Timing

Contamination Delay

- Die **Mindestzeit** zwischen der *Inputänderung* und der **ersten** dadurch verursachten *Outputänderung* wird als **contamination delay** bezeichnet

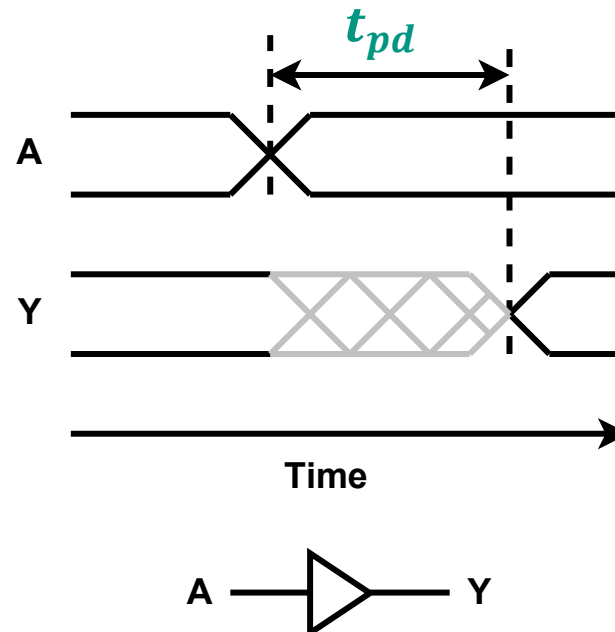


Referenz S. 88 [1]

Timing

Propagation Delay

- Die **maximale Zeit** zwischen der **Inputänderung** und der **endgültigen Outputänderung** wird als **propagation delay** bezeichnet. Der **Ausgang** erreicht seinen **Endwert** innerhalb oder genau bei Ablauf des **propagation delay**

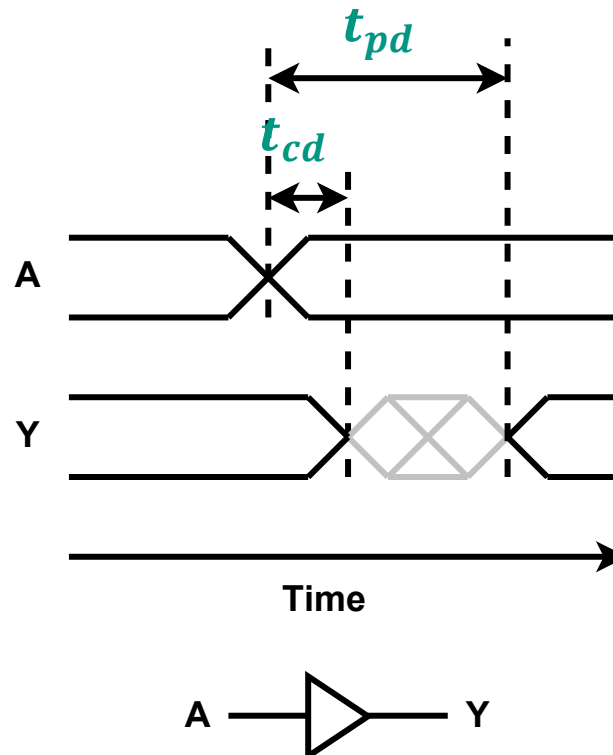


Referenz S. 88 [1]

Timing

Contamination Delay & Propagation Delay

- Die **Zeit zwischen** dem *maximalen* und dem *minimalen* delay ist die Zeit, in der sich das **Ausgangssignal** ändert/stagniert
Der Wert des Signals in dieser Zeit ist nicht von Interesse!



Referenz S. 88 [1]

Timing

Ursache für *delays*

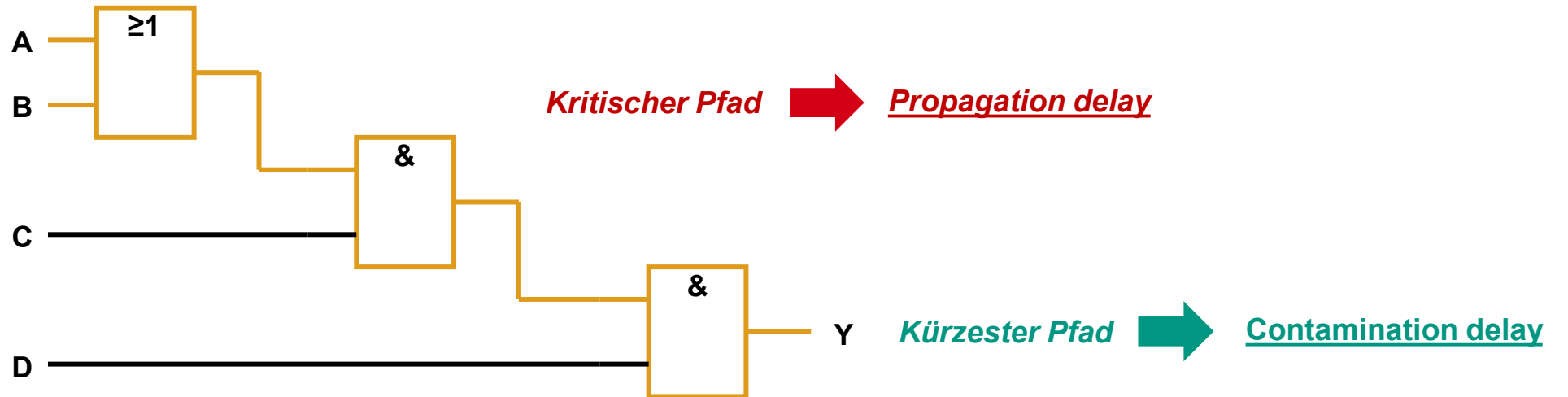
- Die *delays* in **Schaltungen** entstehen vor allem durch die Zeit, die nötig ist, um die **Eingangskapazitäten** der nächsten Stufe zu laden /entladen. Außerdem gibt es eine **physikalische Obergrenze**: die *Signalausbreitungsgeschwindigkeit*
- Warum gibt es Unterschiede *contamination delay* / *propagation delay*?
 - **Unterschiedliche** Verzögerungen für **fallende/steigende Flankenübergänge**
 - Verschiedene **Ein-** und **Ausgänge** eines Systems haben **unterschiedliche *delay*-Beziehungen**:
 - Zwischen dem *frühesten* und *spätesten* Eintreffen entsteht daher ein **Zeitfenster**
 - **Temperaturänderungen** beeinflussen **elektronische Bauteile** und führen zusätzlich zu **Abweichungen**

Siehe S. 88 [1]

Timing

Kritischer Pfad

- Die *delays* werden durch den *Weg* bestimmt, den ein *Signal* vom *Input* zum *Output* nimmt:
 - Der *contamination delay* wird durch den *kürzesten Weg* bestimmt.
 - Der *längste Weg* wird als *kritischer Pfad* (*critical path*) bezeichnet und bestimmt den *propagation delay*

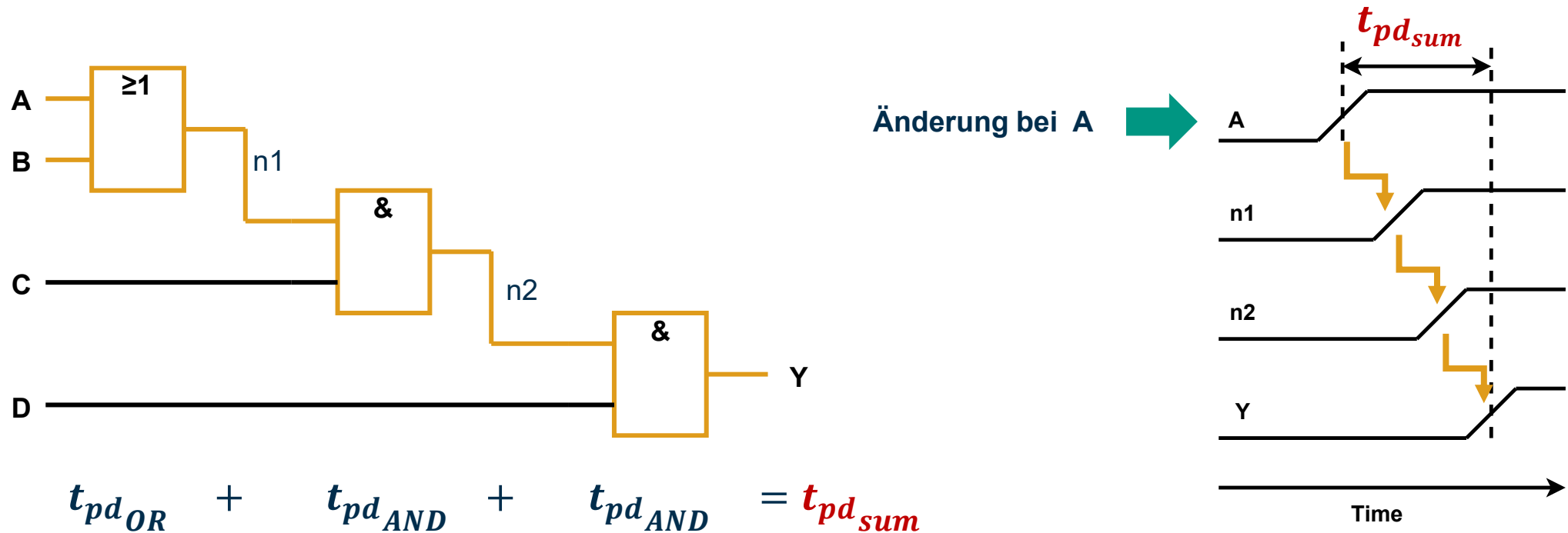


Referenz S. 88 [1]

Timing

Kritischer Pfad

- Der **propagation delay** einer kombinatorischen Schaltung ist die **Summe** der **propagation delays** aller Elemente auf dem **kritischen Pfad**.



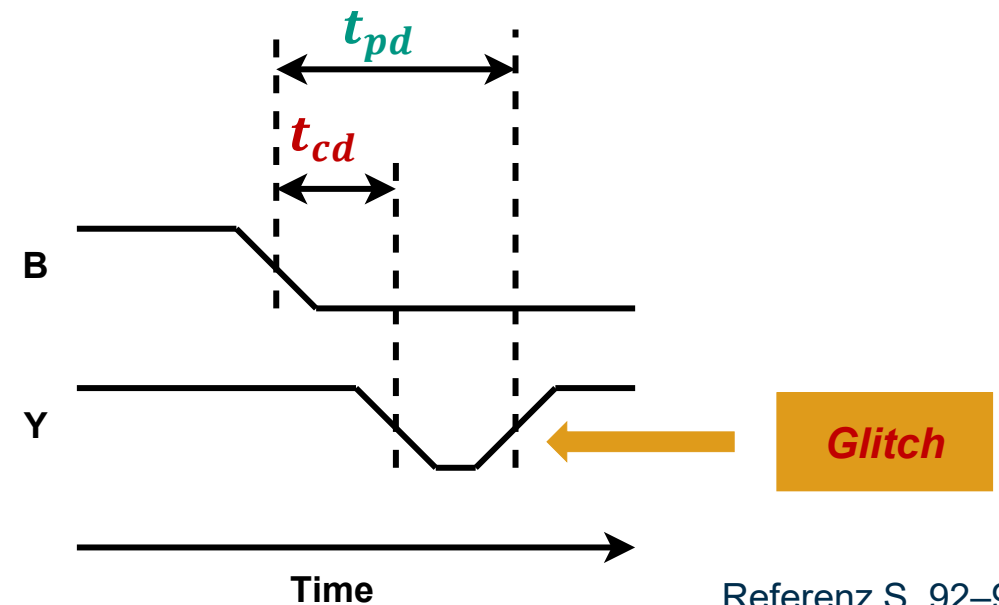
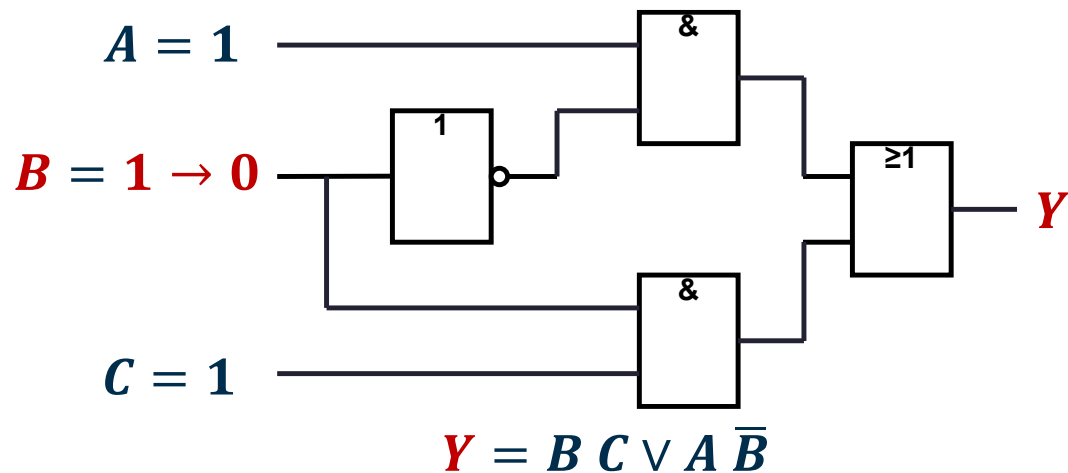
Referenz S. 88 [1]

Timing

Glitch

- Es ist möglich, dass eine **einzelne Inputänderung** aufgrund **unterschiedlicher Pfadlängen mehrere Outputänderungen** verursacht
- Diese werden als **Glitches** oder **Hazards** bezeichnet. Da dies **vor Ablauf des propagation delays** geschieht, verursacht dies in der Regel **keinen Schaden**

Beispiel: **B** wechselt von **1** zu **0**



Referenz S. 92–94 [1]

Timing

Glitch

- Das **KV-Diagramm** kann verwendet werden, um mögliche **Störungen** zu identifizieren
 - Eine **Störung** ist möglich, wenn die **Funktion** ihren **Wert** über verschiedene **Primimplikanten** (**Rechtecke**) hinweg **beibehalten** muss

Beispiel:

B wechselt von **1** zu **0**

A, B		00	01	<u>11</u>	<u>10</u>
C					
0	0 0	0 1	0 5	1 4	
1	0 2	1 3	1 7	1 6	

$$Y = B C \vee A \bar{B}$$

Glitch

Referenz S. 92–94 [1]

Timing

Glitch

- Selbst wenn die **Boolesche Gleichung** korrekt minimiert wurde, können Störungen auftreten
- Man kann eine **Störung vermeiden**, indem man einen **zusätzlichen Implikanten** hinzufügt

Beispiel:

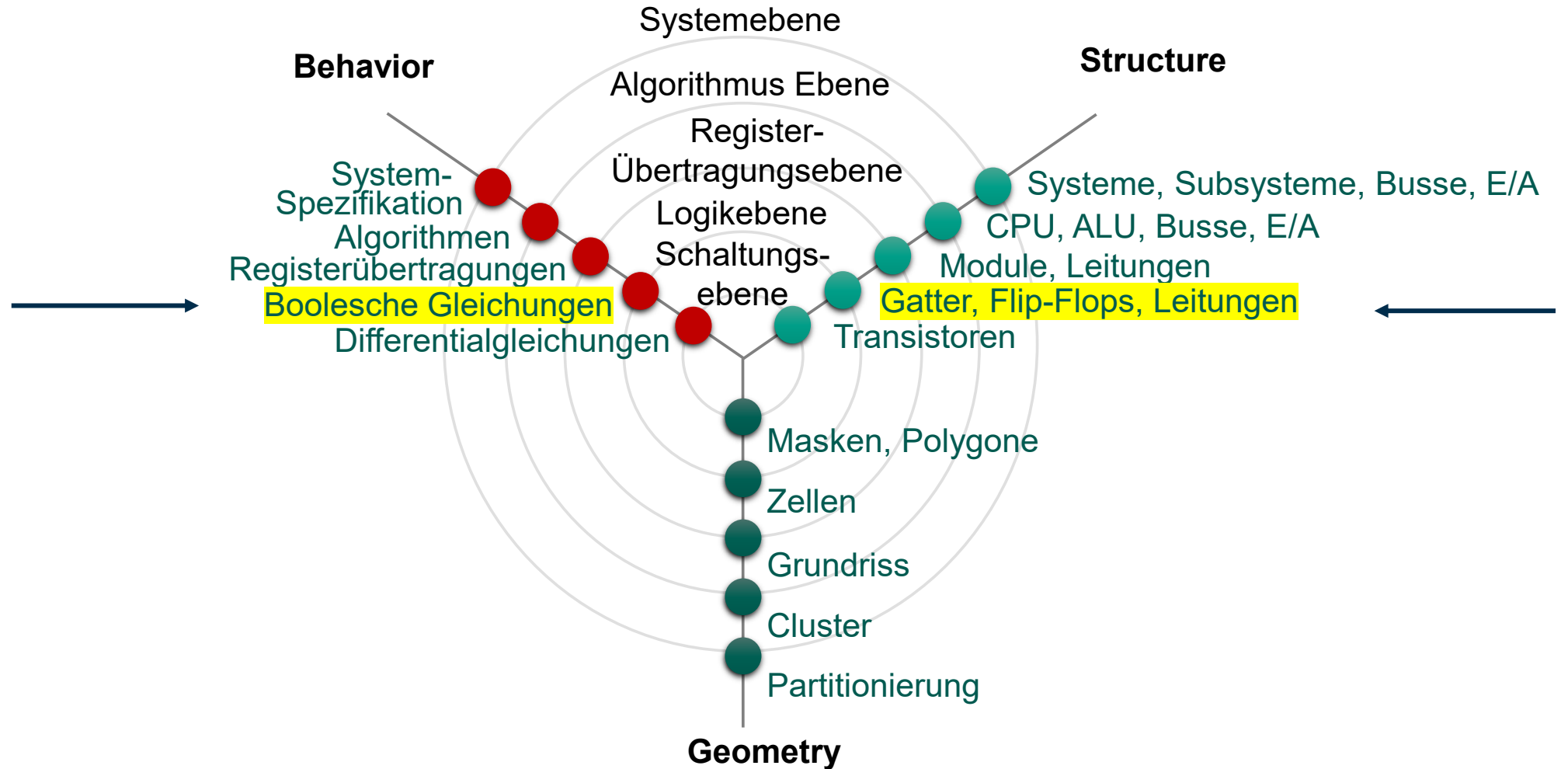
$$Y = B C \vee A \bar{B} \vee A C$$

A, B		00	01	11	10
C					
0		0 0	0 1	0 5	1 4
1		0 2	1 3	1 7	1 6

Referenz S. 92–94 [1]

Advance Organizer

Y-Diagramm:



Referenz

- [1] S. L. Harris und D. M. Harris, *Digital design and computer architecture*, ARM® edition. Waltham, MA: Morgan Kaufmann, 2016
- [2] H. M. Lipp, „Zur Systematik der Diagramm-Minimierung / Methodische Aspekte der Diagramminimierung“, *it – Informationstechnologie*, Band 11, Nr. 1–6, S. 267–271, Dez. 1969
- [3] H. M. Lipp und J. Becker, *Grundlagen der Digitaltechnik*, 7., verb. Aufl. München: Oldenbourg, 2011.

